Projet Zuul : “Henriette’s Feast”

Le festin d'Henriette

Repository GitHub : https://github.com/raphouqld/zuul-good
(Voir commits, ~1 commit par exo)

I. PRÉSENTATION GÉNÉRALE

A. Auteur

QUILLAUD Raphaël

B. Thème

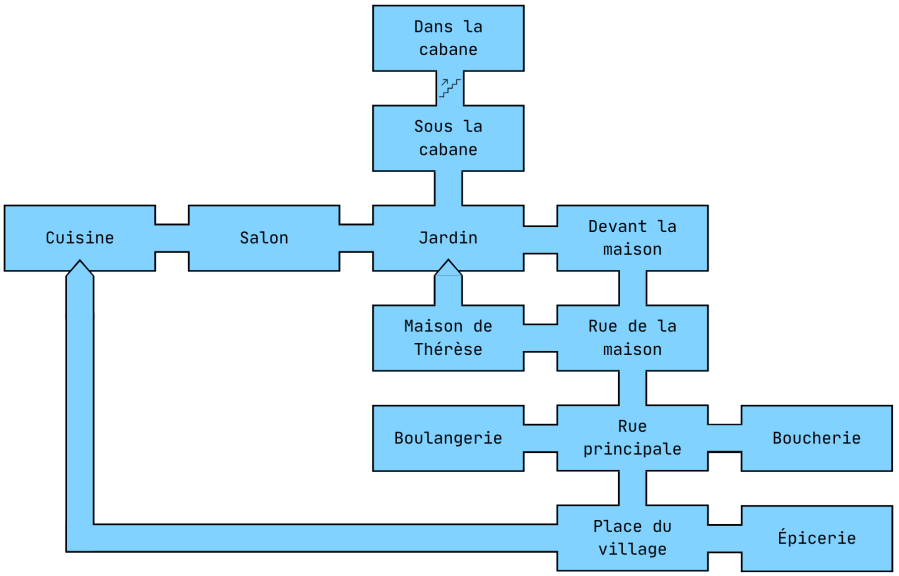
Dans un petit village enneigé, Henriette doit préparer le grand repas de Noël pour la famille, en retrouvant les ingrédients dispersés dans le village.

C. Résumé du scénario

C'est le soir de Noël, dans un petit village. La grand-mère Henriette se rend compte qu'elle a oublié de faire les courses pour le repas de Noël.
Le joueur incarne Henriette et doit parcourir les rues du village, aller chez les voisins et dans les commerces pour récupérer tout ce dont elle a besoin : dinde, bûche, légumes, grand plat, etc.
Le jeu est un jeu d'exploration : le joueur se déplace de pièce en pièce et ramène progressivement les objets dans la cuisine. La partie est gagnée lorsque tous les éléments nécessaires au repas ont été réunis et que Henriette se trouve dans sa cuisine avec ces objets, avant minuit.

D. Plan

Lieux actuellement prévus et déjà codés :



E. Scénario détaillé

Quittez la maison par le jardin.
Allez chez Thérèse pour le plat et mangez le cookie.
Récupérez la dinde (Boucherie) et la farine (Boulangerie).
Achetez le reste à l'Épicerie (Beurre, Sucre, Chocolat, Épices).
Utilisez le raccourci de la Place du Village pour revenir poser vos objets en Cuisine.
Victoire : Tous les objets réunis en cuisine avant minuit

F. Détail des lieux, items, personnages

Lieux

- Cuisine (Kitchen) : Le point de départ (et de fin). C'est ici que Grandma Henriette doit préparer le repas de Noël.
- Salon (Living Room) : Une pièce chaleureuse avec le sapin de Noël, située à l'est de la cuisine.
- Jardin (Garden) : Zone enneigée devant la maison, servant de carrefour vers la cabane, la maison de Thérèse et la rue.
- Cabane des petits-enfants (Tree House) : Située en hauteur (accessible depuis "under the tree house"), elle cache un objet mystérieux (le beamer).
- Devant la maison (Front House) : Portion de rue juste devant la maison d'Henriette.
- Maison de Thérèse (Therese House) : La voisine d'Henriette. On y trouve un grand plat et un délicieux cookie (très lourd !).
- Rue de la maison (House Street) : Relie la zone des habitations à la rue principale.
- Boulangerie (Bakery) : Elle ne fournit pas directement la bûche, mais on y trouve la farine nécessaire.

- Rue principale (Main Street) : Grande rue décorée reliant la boulangerie, la boucherie et la place du village.
- Boucherie (Butcher Shop) : C'est ici que vous trouverez la dinde de Noël.
- Place du village (Main Square) : Lieu central avec un sapin géant, reliant la rue principale, l'épicerie et offrant un raccourci vers la cuisine.
- Épicerie (Grocery) : On y trouve le reste des ingrédients : beurre, sucre, chocolat et épices.

Objets importants

- Dinde (turkey) : Située à la boucherie.
- Ingrédients pour la bûche :
- Farine (flour) : À la boulangerie.
- Beurre (butter), Sucre (sugar), Chocolat (chocolate) : À l'épicerie.
- Épices (spices) : À l'épicerie.
- Grand plat (plate) : Chez Thérèse.
- Livre de recettes (book) : Se trouve initialement dans la cuisine.
- Bûche de bois (log) : Pour la cheminée, située sous la cabane (underTreeHouse).
- Cookie : Offert par Thérèse, il permet de doubler votre capacité de transport si vous le mangez (eat cookie).
- Téléporteur (beamer) : Caché dans la cabane (tree house), il permet de se téléporter après avoir été chargé.

Objectif

- Vous incarnez Henriette. Vous avez 50 coups pour rassembler tous les objets et revenir dans la cuisine avant minuit !

G. Situations gagnantes et perdantes

Victoire : le joueur se trouve dans la cuisine et possède tous les objets indispensables pour le repas de Noël.

Défaite : limite de 50 tours pour l’instant.

H. Énigmes, mini-jeux, combats

Idées d'énigmes simples pour plus tard :

- Les petits-enfants ne donnent un ingrédient qu'après avoir résolu une énigme.
- Thérèse n'accepte de rendre le grand plat qu'après lui avoir rapporté un ingrédient dont elle a besoin.
- Mini-jeux à chaque magasin ?

QUILLAUD Raphaël - E1 - Groupe 9
Projet ZUUL

I. Commentaires

Version actuelle :
- Carte du village créée et fonctionnelle (12 rooms avec connexions).
- Textes d'accueil et d'aide adaptés au jeu.
- Jeu encore minimal : pas d'objets codés, pas d'inventaire, pas d'énigmes programmées.

Partie restante :
- Implémenter les items, l'inventaire, les commandes supplémentaires.

II. RÉPONSES AUX EXERCICES

Ex. 7.5 :

- Création d'une méthode ***printLocationInfo()*** dans Game qui affiche la description de la salle courante (***aCurrentRoom.getDescription()***) et la liste des sorties possibles :

```
private void printLocationInfo() {
   System.out.println("\n" + this.aCurrentRoom.getDescription());
   System.out.print("Exits :");
   if (this.aCurrentRoom.aNorthExit != null) {
       System.out.print(" north");
   }
   if (this.aCurrentRoom.aEastExit != null) {
       System.out.print(" east");
   }
   if (this.aCurrentRoom.aSouthExit != null) {
       System.out.print(" south");
   }
   if (this.aCurrentRoom.aWestExit != null) {
       System.out.print(" west");
   }
   System.out.println();
}
```

- Modification de ***goRoom()*** et ***printWelcome()*** pour qu'elles appellent ***printLocationInfo()***.

- Avant, la situation était inacceptable car il y avait de la duplication de code. En centralisant cet affichage dans ***printLocationInfo()***, toute future modification de la manière de présenter la position du joueur se fera en un seul endroit.

Ex. 7.6 :

- Dans notre projet, passer ***null*** à ***setExits*** signifie volontairement « pas de sortie dans cette direction », donc l'affectation directe (***this.aNorthExit = pNorth;***) suffit, pas besoin de vérifier.
- Ajout dans Room une méthode ***getExit(String direction)*** qui renvoie la salle voisine dans cette direction ou ***null*** si elle n'existe pas.

```
public Room getExit(String pdirection) {
```

```
        return switch (pdirection) {
            case "north" -> this.aNorthExit;
            case "east" -> this.aEastExit;
            case "south" -> this.aSouthExit;
            case "west" -> this.aWestExit;
            default -> null;
        };
    }
```

- Modification de ***goRoom*** dans Game pour appeler ***aCurrentRoom.getExit(vDirection)*** au lieu de tester chaque attribut :

```
    Room vNextRoom = this.aCurrentRoom.getExit(vDirection);
```

- Cela permet de centraliser la gestion des sorties dans Room et facilitera les futures modifications sans changer la logique de Game.

Ex. 7.7 :

- Ajout d'une méthode ***getExitString()*** dans Room qui construit une chaîne décrivant les sorties de la pièce :

```
    public String getExitString() {
        String vExitString = "Exits :";
        if (this.aNorthExit != null) {
            vExitString += " north";
        }
        if (this.aEastExit != null) {
            vExitString += " east";
        }
        if (this.aSouthExit != null) {
            vExitString += " south";
        }
        if (this.aWestExit != null) {
            vExitString += " west";
        }
        return vExitString;
    }
```

- Modification de ***printLocationInfo()*** dans Game pour utiliser cette méthode :

```
    System.out.print(this.aCurrentRoom.getExitString());
```

- Ainsi, c'est ***Room*** qui produit les informations sur ses sorties (elle connaît ses propres exits), et ***Game*** se contente de les afficher au joueur, ce qui sépare la logique de structure des pièces de l'affichage.

Ex. 7.8 :

- Remplacement dans ***Room*** des quatre attributs (***aNorthExit***, ***aEastExit***, ***aSouthExit***, ***aWestExit***) par un attribut ***HashMap<String, Room> aExits*** qui associe chaque direction (clé) à une pièce voisine (valeur).

```
    private final HashMap<String, Room> aExits;
```

- Suppression de l'ancienne méthode ***setExits(...)***, devenue inutile, et remplacement par ***setExit(String pDirection, Room pNeighbor)*** qui ajoute une sortie dans la HashMap.

```
public void setExit(final String pDirection, final Room pNeighbor) {
   this.aExits.put(pDirection, pNeighbor);
}
```

- La méthode ***getExit(String direction)*** renvoie maintenant directement ***aExits.get(direction)*** :

```
public Room getExit(final String pDirection) {
   return this.aExits.get(pDirection);
}
```

- Réécriture de ***getExitString()*** en utilisant ***aExits.keySet()*** et une boucle for-each pour construire la String des sorties à partir des clés de la HashMap :

```
public String getExitString() {
   String vExitString = "Exits :";
   Set<String> vKeys = this.aExits.keySet();

   for (String vDirection : vKeys) {
       vExitString += " " + vDirection;
   }
   return vExitString;
}
```

- Dans ***Game.createRooms()***, adaptation de la création du plan du village pour utiliser ***setExit("direction", room)*** au lieu de ***setExits(north, east, south, west)***.

Ex. 7.8.1 :

- Ajout d'un lieu en hauteur dans le plan : la cabane des petits-enfants (Tree House) ainsi que le lieu situé juste en dessous (Under Tree House).
- Dans ***Game.createRooms()***, modification du jardin pour qu'il mène maintenant vers Under Tree House.
- Ajout des déplacements verticaux :

```
// Under Treehouse
vUnderTreeHouse.setExit("south", vGarden);
vUnderTreeHouse.setExit("up", vTreeHouse);

// Treehouse
vTreeHouse.setExit("down", vUnderTreeHouse);
```

Ex. 7.9 :

- ***keySet()*** retourne un objet de type ***Set<K>*** qui est l'ensemble des clés contenues dans la ***HashMap***, ce qui permet de parcourir facilement toutes les clés avec une boucle ***for-each***.

Ex. 7.10.2 :

- Génération de la Javadoc du projet avec BlueJ et vérification de son contenu dans un navigateur
- La page de la classe ***Game*** ne montre qu'un seul élément dans le résumé (le constructeur ***Game()***), alors que la classe ***Room*** affiche plusieurs méthodes (***getDescription***, ***setExit***, ***getExit***, ***getExitString***, etc.). Cela s'explique par le fait que la Javadoc n'inclus que l'interface publique des classes : dans ***Game***, presque toutes les méthodes sont ***private*** et ne sont donc pas visibles dans la documentation, tandis que Room a plusieurs méthodes ***public***.

Ex 7.11 :

- Modification de la classe Game pour que le constructeur initialise seulement les salles et le parser, et la méthode ***play()*** est maintenant ***public***.
- Ajout dans Room de la méthode ***getLongDescription()*** qui renvoie une description détaillée de la pièce incluant la liste des sorties :

```
    public String getLongDescription() {
        return "You are " + this.aDescription + ".\n" + this.getExitString();
    }
```

Ex. 7.14 :

- Ajout de la commande ***look()***, qui affiche la description détaillée de la pièce courante via ***Room.getLongDescription()*** :

```
    private void look() {
        System.out.println(this.aCurrentRoom.getLongDescription());
    }
```

- Si le joueur saisit un second mot, le jeu répond qu'il ne sait pas regarder quelque chose en particulier (**OPTIONNEL**).

Ex. 7.15 :

- Ajout de la commande ***eat*** dans la classe ***Game***, qui affiche un message indiquant que le personnage a mangé et n'a plus faim, sans autre effet pour l'instant :

```
    private void eat() {
        System.out.println("You have eaten now and you are not hungry any
more.");
    }
```

- Ajout de ***eat*** à la liste des commandes valides dans ***CommandWords*** et adapté ***processCommand*** pour qu'elle reconnaisse et exécute cette nouvelle commande.

Ex. 7.16 :

- Amélioration de l'affichage de la liste des commandes dans l'aide. Ajout d'une méthode ***showAll()*** dans ***CommandWords***, qui parcourt les commandes valides et les affiche :

```
    public void showAll() {
        for (String vCommand : this.aValidCommands) {
            System.out.println(vCommand + " ");
        }
        System.out.println();
    }
```

- Ajout d'un accesseur ***getCommandWords()*** dans ***Parser***, permettant à ***Game*** d'appeler ***aParser.getCommandWords().showAll()*** dans ***printHelp*** :

```
private void printHelp() {
   System.out.println("""
           You are Grandma Henriette in a small snowy village.
           You must find everything you need for the Christmas dinner.

           Your command words are:""");
   this.aParser.getCommandWords().showAll();
}
```

Ex. 7.17 :

- L'ajout d'une nouvelle commande dans ***CommandWords*** ne nécessite plus de modification de ***Game*** pour l'affichage de l'aide, car ***printHelp*** utilise maintenant ***aParser.getCommandWords().showAll()***.
- En revanche, une modification de ***Game*** reste nécessaire pour définir cette nouvelle commande dans ***processCommand***.

Ex 7.18 :

- Amélioration de l'affichage de l'aide : ajout d'une méthode ***getCommandList()*** dans ***CommandWords*** qui construit une ***String*** contenant les commandes valides :

```
public String getCommandList() {
   String vCommands = "";
   for (int vI = 0; vI < this.aValidCommands.length; vI++) {
       vCommands = vCommands + this.aValidCommands[vI] + " ";
   }
   return vCommands.trim();
}
```

- Puis d'une méthode ***getCommandList()*** dans Parser qui se contente de relayer cette chaîne. La méthode ***printHelp()*** de ***Game*** affiche maintenant le résultat de ***aParser.getCommandList()*** :

```
public String getCommandList() {
   return this.aValidCW.getCommandList();
}
```

Ex. 7.18.1 :

- Comparaison du projet avec l'exemple **zuul-better** : Suppression de la méthode inutile **showAll()** dans **CommandWords** afin de se conformer à la structure de zuul-better.

Ex. 7.18.2 :

- Intégration de **StringBuilder** dans la méthode **Room.getExitString()** afin d'optimiser la construction de la chaîne des sorties dans une boucle.

Ex. 7.18.3 :

- Génération par Intelligence Artificielle des images correspondant à chaque pièce du jeu : voir le dossier “img” du projet sur GitHub : https://github.com/raphouqld/zuul-good/tree/main/img

Ex. 7.18.4 :

- Choix du titre définitif du jeu : ***Henriette's Feast***. Intégration de ce titre dans le message de bienvenue affiché au lancement du jeu, dans le rapport et sur la page web de présentation du projet.

Ex 7.18.5 :

- Ajout d’un attribut **HashMap<String, Room>** dans la classe **Game** pour référencer toutes les pièces du village par un identifiant texte. La map est initialisée dans le constructeur et remplie dans **createRooms** :
  - Attribut :

```
private HashMap<String, Room> aRooms;
```

  - Constructeur :

```
this.createRooms();
```

  - createRooms() :

```
this.aRooms.put("kitchen", vKitchen);
this.aRooms.put("livingRoom", vLivingRoom);
this.aRooms.put("garden", vGarden);
this.aRooms.put("underTreeHouse", vUnderTreeHouse);
this.aRooms.put(...);
```

Ex. 7.18.6 :

- Passage à une architecture graphique, inspirée du projet **zuul-with-images**. Étude de la javadoc des classes **Game**, **GameEngine**, **UserInterface** et **Parser**, puis du **.jar** fourni, afin de comprendre le fonctionnement de l’interface graphique.
- Adaptation de **Room** pour gérer une image associée à chaque lieu (attribut **aImageName**, constructeur **Room(String, String)** et méthode **getImageName()**), avec mise à jour de **createRooms** pour fournir le chemin d’image correspondant à chaque pièce du village enneigé.
- Remplacement de **Parser** par une version prenant une **String** en paramètre et utilisant **StringTokenizer** au lieu de **Scanner**, car la saisie n’est plus lue dans le terminal mais transmise par l’interface graphique (**UserInterface**).
- Refactorisation de **Game** en simple classe de lancement qui instancie **GameEngine** et **UserInterface**, puis les relie via **setGUI**. La logique de jeu (création des salles, commandes **go**, **help**, **look**, **eat**, …) est désormais centralisée dans **GameEngine**, où **processCommand** a été remplacée par **interpretCommand(String)** :

```
public class Game {
   private UserInterface aGui;
   private GameEngine aEngine;

   /**
```

```
     * Creates the game, initializes the rooms and the parser,
     * then starts the game loop.
     */
    public Game() {
        this.aEngine = new GameEngine();
        this.aGui = new UserInterface(this.aEngine);
        this.aEngine.setGUI(this.aGui);
    }

    public static void main(String[] args) {
        new Game();
    }
} // Game
```

- Intégration de la nouvelle classe **UserInterface** (fenêtre Swing avec zone de texte, champ de saisie et zone d'image) et remplacement de tous les affichages destinés au joueur (**System.out**) par des appels à **aGui.print()**, **aGui.println()** et **aGui.showImage()**. La méthode **printLocationInfo** a été conservée et complétée pour afficher à la fois la description, les sorties de la salle et l'image correspondante. Les imports génériques * ont été remplacés par des imports explicites pour les classes Swing et AWT utilisées :

```
import java.awt.BorderLayout;
import java.awt.Dimension;

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.net.URL;

import javax.swing.ImageIcon;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
```

Ex. 7.18.8 :

- Ajout d'un bouton help dans l'interface graphique **UserInterface**, créé avec **JButton("help")**. Cette modification permet de déclencher une commande du jeu directement depuis l'interface graphique :

```
this.aHelpButton = new JButton("help");
vCenterPanel.add(this.aHelpButton, BorderLayout.EAST);
```

- Ajout d'un écouteur d'action sur le bouton avec **addActionListener(this)**, en complément de celui déjà présent sur **aEntryField**. La méthode **actionPerformed(ActionEvent)** a alors été adaptée pour distinguer l'origine de l'événement grâce à **getSource()** :

```
@Override
public void actionPerformed(final ActionEvent pE) {
    if (pE.getSource() == this.aHelpButton) {
        this.aEngine.interpretCommand("help");
```

```
        }
        else if (pE.getSource() == this.aEntryField) {
            this.processCommand();
        }
    }
```

Ex. 7.19.2 :

- Toutes les images ont été déplacées du dossier img au dossier Images, et le chemin d'accès dans **createRooms()** a été mis à jour :

```
    Room vKitchen       = new Room("in Henriette's warm, cosy
kitchen","Images/kitchen.jpg");
    Room vLivingRoom    = new Room("in the living room, next to the Christmas
tree","Images/livingRoom.jpg");
    Room vGarden        = new Room("in the snowy front garden of the
house","Images/garden.jpg");
    Room vUnderTreeHouse = new Room("under the tree house, in the
snow","Images/underTreeHouse.jpg");
    Room vTreeHouse     = new Room("in the children's tree
house","Images/treeHouse.jpg");
    Room ...
```

Ex. 7.20 - 7.21 :

- Ajout d'une classe **Item** représentant un objet du jeu, défini par une description et un poids. La classe **Room** a été étendue pour contenir au plus un item (attribut **aItem**, méthodes **setItem**, **getItem** et **getItemString**), et sa description longue affiche désormais, en plus des sorties, soit l'objet présent soit le message *No item here*. Dans **GameEngine**, plusieurs items liés au repas de Noël (farine, beurre, dinde, épices, bûche de bois) sont créés dans **createRooms()** et placés dans les pièces du village, que le joueur découvre en entrant dans ces salles.

Ex 7.22 :

- Extension de la classe **Room** pour qu'une pièce puisse désormais contenir plusieurs objets au lieu d'un seul. L'attribut unique **aItem** a été remplacé par une collection **HashMap<String, Item> aItems**, permettant de stocker plusieurs items identifiés par leur nom :

```
    public class Room {
        private final String aDescription;
        private final HashMap<String, Room> aExits;
        private final String aImageName;
        private HashMap<String, Item> aItems;
    ...
    }
```

- La méthode **setItem(...)** a été remplacée par **addItem(Item)**, et une méthode **getItem(String)** permet de retrouver un objet à partir de son nom, tandis que **getItemString()** parcourt l'ensemble de la collection pour afficher la liste complète des items présents :

```
    public void addItem(final Item pItem) {
        this.aItems.put(pItem.getName(), pItem);
    }

    public Item getItem(final String pItemName) {
```

```
        return this.aItems.get(pItemName);
    }

    public String getItemString() {
        if (this.aItems.isEmpty()) {
            return "No item here.";
        }
        StringBuilder vItems = new StringBuilder("Items : ");
        for (Item vItem : this.aItems.values()) {
            vItems.append("\n- ").append(vItem.getLongDescription());
        }
        return vItems.toString();
    }
```

- La classe **Item** a été enrichie d'un attribut **aName** en plus de la description et du poids.
- Enfin, **createRooms()** dans **GameEngine** a été modifiée pour créer plusieurs objets nécessaires au repas de Noël (farine, beurre, sucre, chocolat, dinde, épices, bûche de bois) et en déposer plusieurs dans certaines pièces :

```
    // Items
    Item vFlour = new Item("flour","a bag of flour", 1);
    Item vButter = new Item("butter","a block of butter", 1);
    Item vSugar = new Item("sugar", "a packet of sugar", 1);
    Item vChocolate = new Item("chocolate", "a bar of dark chocolate", 1);
    Item vTurkey = new Item("turkey","a beautiful Christmas turkey", 5);
    Item vSpices = new Item("spices","a small box of Christmas spices", 1);
    Item vWoodLog = new Item("log","a dry log for the fireplace", 2);

    // Place items in rooms
    vBakery.addItem(vFlour);
    vGrocery.addItem(vButter);
    vButcherShop.addItem(vTurkey);
    vThereseHouse.addItem(vSpices);
    vUnderTreeHouse.addItem(vWoodLog);
    vBakery.addItem(vSugar);
    vBakery.addItem(vChocolate);
```

Ex.7.22.2 :

- Intégration des objets du scénario dans les différentes pièces du jeu. Un **item** a été ajouté dans la pièce initiale du jeu, la cuisine d'Henriette **vKitchen**, conformément à l'énoncé. Par ailleurs, certaines salles contiennent désormais plusieurs objets, notamment le magasin.

Ex. 7.23 :

- Ajout de la commande ***back***, qui permet au joueur de revenir dans la salle où il se trouvait juste avant. Pour cela, **GameEngine** mémorise la salle précédente dans un nouvel attribut **aPreviousRoom**, mis à jour à chaque déplacement, et une méthode **back()** échange la salle courante avec cette salle précédente avant d'afficher à nouveau les informations de lieu. La commande ***back*** a été ajoutée à la liste des mots reconnus par le parseur et traitée dans **interpretCommand(...)**.

Ex. 7.26 :

- Modification de la commande ***back*** afin qu'elle puisse être utilisée plusieurs fois de suite, jusqu'au début du jeu. L'attribut **aPreviousRoom** a été remplacé par une pile

aPreviousRooms de type **Stack<Room>**, qui mémorise tout l'historique des salles traversées. À chaque déplacement dans **goRoom(...)**, la salle courante est ajoutée à la pile avec **push**, puis la commande ***back*** récupère la dernière salle enregistrée avec **pop** pour y ramener le joueur. Un test sur **empty()** a également été ajouté afin de gérer correctement le cas où l'on est revenu au point de départ du jeu et qu'aucun retour supplémentaire n'est possible.

Ex. 7.27 :

- Tests de base envisagés pour ***Henriette's Feast*** :
  - Test de démarrage :
    Vérifier que le jeu commence bien dans la cuisine d'Henriette (**vKitchen**) et que la description de la salle s'affiche correctement.
  - Test de navigation :
    Vérifier que la commande **go** déplace bien le joueur dans la bonne salle (ex. **go north** depuis la cuisine mène bien au salon), et qu'une direction invalide affiche un message d'erreur sans changer la salle courante.
  - Test des commandes :
    Vérifier que **help** affiche la liste des commandes, que **quit** termine le jeu, et qu'une commande inconnue produit un message d'erreur sans plantage.
  - Test des **items** :
    Vérifier que les objets placés dans les pièces (farine, dinde, bûche…) sont bien listés dans la description longue de la salle via **getLongDescription()**.
  - Test de la commande **back** :
    Vérifier que **back** ramène bien le joueur dans la salle précédente, et que plusieurs **back** successifs remontent correctement la pile **aPreviousRooms**.
  - Test de **look** :
    Vérifier que la commande **look** réaffiche correctement la description complète de la salle courante, sorties et items inclus.

Ex. 7.28.1 :

- Ajout de la commande **test** dans **CommandWords.java**.

- Implémentation de la méthode **test(Command pCommand)** dans **GameEngine.java**. Elle récupère le second mot de la commande (le nom du fichier sans extension), lui ajoute ***.txt***, puis ouvre ce fichier avec un **Scanner**. Chaque ligne non vide est passée à **interpretCommand()**, simulant ainsi une saisie manuelle de l'utilisateur :

```
    private void test(final Command pCommand) {
        if (!pCommand.hasSecondWord()) {
            this.aGui.println("Test which file ? Usage : test <filename>");
            return;
        }

        String vFileName = pCommand.getSecondWord() + ".txt";

        try {
            Scanner vScanner = new Scanner(new File(vFileName));
            while (vScanner.hasNextLine()) {
                String vLine = vScanner.nextLine().trim();
                if (!vLine.isEmpty()) {
                    this.interpretCommand(vLine);
```

```
                }
            }
            vScanner.close();
        } catch (FileNotFoundException e) {
            this.aGui.println("Test file not found : " + vFileName);
        }
    }
```

- Si le fichier n'est pas trouvé, un message d'erreur est affiché. Si aucun second mot n'est fourni, un message d'aide est affiché.

Ex. 7.28.2 :

- **court.txt** : fichier court de test comportant 6 commandes différentes (**help**, **go**, **look**, **back**, **eat**, **quit**), permettant de vérifier que la commande test fonctionne correctement :

```
help
go east
look
back
eat
quit
```

- **parcours.txt** : suite minimale de commandes permettant de parcourir toutes les salles du village dans l'ordre le plus direct. Ce fichier sera mis à jour lorsque la commande take et la condition de victoire seront implémentées.
- **exploration.txt** : exploration exhaustive du jeu, incluant toutes les salles accessibles, les commandes d'information (**help**, **look**, **eat**), les déplacements inutiles et l'utilisation de **back**.

Ex. 7.29 :

- Création de la classe **Player** qui encapsule les informations propres au joueur : son nom (**aName**), sa salle courante (**aCurrentRoom**) et son historique de navigation (**aPreviousRooms**). La pile **Stack<Room>** et les méthodes associées (**goTo()**, **goBack()**) migrent de **GameEngine** vers **Player**.
- Dans **GameEngine**, les attributs **aCurrentRoom** et **aPreviousRooms** sont remplacés par un unique attribut **aPlayer** de type **Player**, instancié à la fin de **createRooms()**.
- La méthode **goRoom()** est réorganisée en 3 parties distinctes :
    1. **GameEngine** interprète la commande et détermine la destination
    2. **Player** effectue le déplacement via **goTo()**
    3. **GameEngine** affiche les informations de la nouvelle salle
- La méthode **back()** délègue maintenant le déplacement à **Player.goBack()** qui retourne un booléen indiquant si le retour est possible.

Ex. 7.30 :

- Ajout d'un attribut **aItem** (de type **Item**) dans la classe **Player** pour stocker l'objet porté par le joueur (*null* si rien). Les méthodes **pickUp(Item)**, **drop()**, **hasItem()** et **getItem()** gèrent cet état.

- Ajout de **getItem(String)** et **removeItem(String)** dans **Room** pour permettre de chercher et retirer un *item* par son nom. Les clés du **HashMap** sont stockées en minuscules via **pItem.getName().toLowerCase()** dans **addItem()**, et la saisie du joueur est normalisée en minuscules dans **take()**, afin de rendre la commande insensible à la casse.
- Implémentation des commandes **take** et **drop** dans **GameEngine** : **take** vérifie que le joueur ne porte rien, cherche l'item dans la salle courante et le transfère au joueur via **pickUp()** ; **drop** dépose l'item porté dans la salle courante via **addItem()**.

Ex. 7.31 :

- Remplacement de l'attribut **aItem** (un seul Item) par un **HashMap<String, Item> aItems** dans **Player**, suivant le même principe que **Room**. La collection n'est pas accessible depuis l'extérieur — seules les méthodes **pickUp()**, **drop(String)**, **hasItem(String)**, **hasItems()** et **getInventoryString()** exposent les opérations nécessaires.
- **drop()** accepte désormais un nom d'item en paramètre, permettant de déposer un item spécifique parmi ceux portés.
- Ajout de la commande **inventory** pour que le joueur puisse consulter ce qu'il porte.

Ex. 7.31.1 :

- Création de la classe **ItemList** qui encapsule un **HashMap<String, Item>** et expose les méthodes **addItem()**, **getItem()**, **removeItem()**, **contains()**, **isEmpty()**, **getItemString()** et **getInventoryString()**. La normalisation en minuscules est centralisée dans **ItemList**, ce qui la retire de **Room** et **Player** :

```
import java.util.HashMap;

public class ItemList {
   private HashMap<String, Item> aItems;

   public ItemList() {
       this.aItems = new HashMap<String, Item>();
   }

   public void addItem(final Item pItem) {
       this.aItems.put(pItem.getName().toLowerCase(), pItem);
   }

   public Item getItem(final String pName) {
       return this.aItems.get(pName.toLowerCase());
   }

   public Item removeItem(final String pName) {
       return this.aItems.remove(pName.toLowerCase());
   }

   public boolean contains(final String pName) {
       return this.aItems.containsKey(pName.toLowerCase());
   }

   public boolean isEmpty() {
       return this.aItems.isEmpty();
   }

   public String getItemString() {
       if (this.aItems.isEmpty()) return "No item here.";
       StringBuilder vResult = new StringBuilder("Items :");
```

```
        for (Item vItem : this.aItems.values()) {
            vResult.append("\n- ").append(vItem.getLongDescription());
        }
        return vResult.toString();
    }

    public String getInventoryString() {
        if (this.aItems.isEmpty()) return "You are not carrying anything.";
        StringBuilder vResult = new StringBuilder("Inventory :");
        for (Item vItem : this.aItems.values() {
            vResult.append("\n- ").append(vItem.getLongDescription());
        }
        return vResult.toString();
    }
}
```

- **Room** et **Player** remplacent leur **HashMap<String, Item>** par un attribut **ItemList** et délèguent toutes les opérations sur les items à cette classe. Leurs méthodes publiques restent inchangées.

Ex. 7.32 :

- Ajout de deux attributs dans Player : **aMaxWeight** (poids maximum que le joueur peut porter, passé en paramètre du constructeur) et **aCurrentWeight** (poids actuellement porté, mis à jour à chaque **pickUp()** et **drop()**).
- **pickUp()** retourne désormais un booléen : ***false*** si l'ajout de l'item dépasse le poids maximum, ***true*** sinon. **GameEngine** affiche un message d'erreur en cas de refus, ainsi que le poids actuel/maximum via **getWeightString()**.
- Le poids maximum transportable par Henriette est fixé à 10. Les items légers (poids 1) sont facilement transportables, mais la dinde (poids 5) occupe une grande partie de la capacité, ce qui force le joueur à planifier ses déplacements.

Ex. 7.33 :

- La commande **inventory** existait déjà, elle a été renommée en **items**. Elle affiche désormais également le poids total porté via **getWeightString()**, indiquant le poids courant et le poids maximum supporté par le joueur.

```
    case "items" :
        this.aGui.println(this.aPlayer.getInventoryString());
        this.aGui.println(this.aPlayer.getWeightString());
        break;
```

Ex. 7.34 :

- La commande **eat** est étendue pour accepter un second mot (le nom de l'item à manger). Si l'*item* n'est pas dans l'inventaire, un message d'erreur est affiché. Seul le cookie peut être mangé : il est retiré de l'inventaire via **drop()** et le poids maximum du joueur est doublé via **setMaxWeight()**. Manger tout autre *item* est refusé avec un message d'avertissement. L'*item* spécial est un cookie mystérieux laissé par la voisine Thérèse, avec un indice dans la description de sa maison.

Ex. 7.42 :

- Ajout d'une limite de temps sous forme de compteur de coups dans **Player** (**aMoveCount**, **aMaxMoves = 50**). Chaque commande de jeu (hors ***help***, ***quit*** et

test) incrémente le compteur via **incrementMoves()**. Si le compteur atteint la limite, **isOutOfTime()** retourne *true*, un message de game over s'affiche et le jeu se termine via **endGame()**. Un avertissement est affiché lorsqu'il reste exactement 10 coups. Le nombre de coups restants est affiché à chaque **printLocationInfo()** via **getRemainingMoves()**.

Ex. 7.43 :

- Une trap door est implémentée comme une sortie à sens unique : on appelle **setExit()** uniquement dans un sens, sans sortie retour. Deux trap doors sont présentes dans le jeu (voir plan).

Ex. 7.44 :

- Création d'une classe **Beamer** héritant de **Item**, qui stocke une **Room** chargée (**aChargedRoom**). **charge(Room)** mémorise la salle courante, **fire()** retourne cette salle et remet **aChargedRoom** à ***null*** (réutilisable après recharge obligatoire). La salle est stockée dans le **Beamer** lui-même afin de supporter plusieurs téléporteurs ou plusieurs joueurs sans modification. Deux commandes **charge** et **fire** sont ajoutées dans **GameEngine**, qui récupèrent le **beamer** depuis l'inventaire via **getItem()** (sans le retirer) et vérifient via **instanceof** que l'item est bien un **Beamer**. Le *beamer* est placé dans la cabane dans les arbres et peut être ramassé, chargé dans une autre pièce, puis déclenché dans une troisième.

III. MODE D'EMPLOI

<u>Pour utiliser Henriette's Feast :</u>
- Lancer la classe Game.
- Lire le message d'accueil, puis saisir les commandes dans le champ de texte en bas de la fenêtre. Le bouton help permet aussi d'afficher l'aide directement.
- Se déplacer avec la commande go suivie d'une direction : north, south, east, west, up ou down, selon les sorties disponibles dans la salle courante.
- Utiliser look pour réafficher la description complète du lieu, ses sorties, les objets présents et le nombre de coups restants.
- Utiliser back pour revenir sur ses pas.
- Utiliser take <nomObjet> pour ramasser un objet, drop <nomObjet> pour le reposer, et items pour afficher l'inventaire ainsi que le poids transporté.
- Utiliser eat <nomObjet> pour tenter de manger un objet. Seul le cookie a un effet spécial : il augmente la capacité de transport.
- Utiliser charge et fire après avoir récupéré le beamer pour enregistrer une salle puis s'y téléporter plus tard.
- Utiliser test <nomFichier> pour exécuter automatiquement les commandes contenues dans un fichier texte de test.
- Utiliser quit pour fermer la partie.

<u>But du jeu :</u>
- Explorer le village, récupérer les objets utiles au repas de Noël et les transporter en respectant la limite de poids.

- Faire attention au nombre de coups restant : la partie se termine si la limite est atteinte.

IV. DÉCLARATION ANTI-PLAGIAT

Je soussigné, Raphaël Quillaud, certifie que ce rapport et le code associés sont le fruit de mon travail personnel et ne comportent aucun plagiat ni copie non autorisée.
Les éventuelles sources d'inspiration ou d'aide (documentation officielle, cours, supports pédagogiques) ont été utilisées uniquement pour comprendre les concepts et non pour reproduire du code ou du texte à l'identique.